Name: Ken Andrew B. Francisco **Student Number:** 202301594

Course, Year & Section: BSIT 4-5 **Date:** Sept 17, 2026

ONLINE ACTIVITY #2

Personalized Linux, and Git Cheat Sheet

1. Basic Linux Commands

Run `man <command>` or `<command> --help` for the complete list of options.

Command	What it does	Example
`pwd`	Prints the full path of the directory you are currently in.	`$ pwd`
`ls`	Lists files and folders; `-a` reveals hidden ones, `-l` adds details.	`$ ls -la`
`cd`	Changes the working directory; `cd ..` goes up one level.	`$ cd ~/projects`
`mkdir`	Creates a new folder; `-p` auto-creates any missing parent directories.	`$ mkdir -p site/css`
`touch`	Creates a blank new file, or updates the timestamp on an existing one.	`$ touch index.html`
`cp`	Copies a file to another location; use `-r` to copy an entire folder.	`$ cp -r site backup`
`mv`	Moves a file or folder to a new path, or renames it if the path stays the same.	`$ mv old.txt new.txt`

Command	What it does	Example
`rm`	Deletes a file or directory (`-r`). This is permanent — no trash bin.	`$ rm -r old_build`
`cat`	Reads a file and prints all of its content straight to the terminal.	`$ cat notes.txt`
`echo`	Outputs a line of text; redirect with `>` to write it into a file.	`$ echo "Hi" > hi.txt`
`grep`	Scans files for a text pattern; `-r` searches folders, `-n` shows line numbers.	`$ grep -rn "error" .`
`chmod`	Adjusts a file's permissions; `+x` makes it runnable as a script.	`$ chmod +x run.sh`
`sudo`	Executes the next command as a superuser (administrator).	`$ sudo apt update`
`man`	Opens the manual page for a command; press `q` when done reading.	`$ man grep`

2. Basic Git Commands

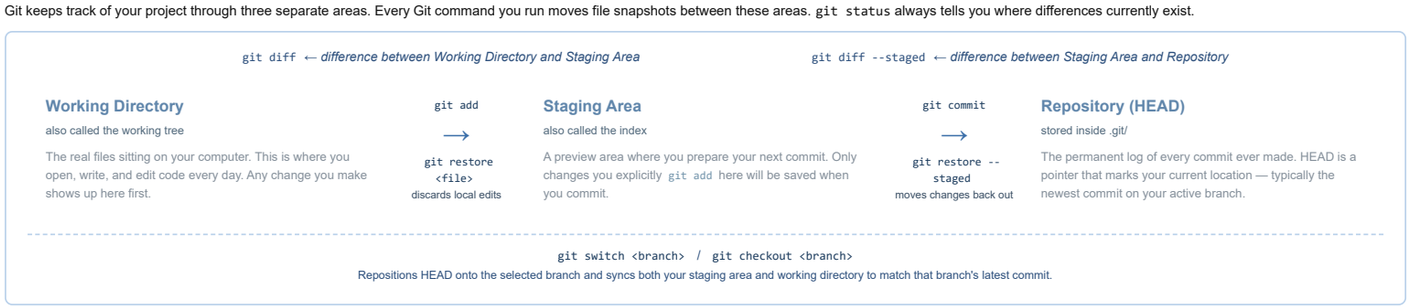
Commands grouped by purpose | Use `git <command> --help` to read the full documentation.

Command	What it does	Example usage	Useful options / flags
▶ Get a repo			
`git init`	Turns the current folder into a Git repository by generating a hidden `.git` directory that stores all version history.	`$ git init -b main`	`-b <name>` name the default branch \| `--bare` create a repo with no working directory (for servers)
`git clone`	Downloads a complete copy of an existing repository — code plus history — and records the source URL as *origin*.	`$ git clone <url>`	`--branch <name>` start on a specific branch \| `--depth 1` fetch only the latest commit \| `--single-branch` skip all other branches
▶ Record changes			
`git status`	Reports which branch is active and which files are new, edited, or already staged for the next commit.	`$ git status -s`	`-s` compact one-line output per file \| `-b` show branch name in short mode \| `--ignored` include files that Git is ignoring
`git add`	Queues specific file changes into the staging area so they will be saved in the next commit.	`$ git add index.html` `$ git add .`	`.` stage all changes in this directory \| `-A` include deletions \| `-p` choose individual chunks interactively \| `-u` only already-tracked files
`git diff`	Shows exactly which lines were added or removed in files that have been edited but not yet staged.	`$ git diff` `$ git diff --staged`	`--staged` compare staged changes vs. last commit \| `--stat` show a file-level summary instead \| `main dev` diff between two branches
`git commit`	Saves everything in the staging area as a permanent snapshot in the project history, along with a message.	`$ git commit -m "Add nav bar"`	`-m "msg"` write the message inline \| `-a` auto-stage all tracked changes first \| `--amend` rewrite the previous commit
▶ History & undo			
`git log`	Displays the commit history from most recent to oldest, showing each commit's ID, author, date, and message.	`$ git log --oneline --graph`	`--oneline` one line per commit \| `--graph` show branch/merge tree \| `--all` show all branches \| `-n 5` limit to 5 commits
`git restore`	Reverts a file to its last committed state, or removes it from the staging area without losing edits. **Discarding working-tree changes is irreversible.**	`$ git restore style.css` `$ git restore --staged app.js`	`--staged` unstage without discarding edits \| `--worktree` throw away working-directory changes \| `--source=<SHA>` pull from a specific commit
`git tag`	Marks a commit with a named label such as *v1.0*, often used to indicate a release point. Push tags with `git push --tags`.	`$ git tag -a v1.0 -m "Release"`	`-a` annotated tag (stores author & message) \| `-m` inline message \| `-l "v1.*"` filter by pattern \| `-d` delete a tag
▶ Branching			
`git branch`	Lists all local branches, or creates and deletes them. Note: creating a branch does not switch you to it automatically.	`$ git branch feature-nav`	`-a` show remote branches too \| `-m` rename the active branch \| `-d` delete a merged branch \| `-D` force-delete \| `-v` show last commit per branch
`git switch` / **`git checkout`**	Moves you onto a different branch and updates your working files to match. *switch* is the newer branch-only version; *checkout* also handles file restores.	`$ git switch feature-nav` `$ git checkout -b feature-nav`	`-c <name>` / `-b <name>` create and switch in one step \| `-` return to the previous branch \| `--orphan <name>` new branch with no history
`git merge`	Brings in all commits from another branch and incorporates them into the branch you are currently on.	`$ git switch main` `$ git merge feature-nav`	`--no-ff` always create a merge commit \| `--squash` combine all commits into one \| `--abort` stop a merge that ran into conflicts

Setting up Git for the first time? Configure your identity before committing: `git config --global user.name "Your Name"` & `git config --global user.email "you@example.com"`

3. The Three-Tree Architecture of Git

Git keeps track of your project through three separate areas. Every Git command you run moves file snapshots between these areas. `git status` always tells you where differences currently exist.



References

Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. https://git-scm.com/book/en/v2

Git SCM. (n.d.). *Git reference documentation*. Retrieved September 2026, from https://git-scm.com/docs

Free Software Foundation. (n.d.). *GNU Coreutils manual*. https://www.gnu.org/software/coreutils/manual/

tldr-pages contributors. (n.d.). *tldr pages*. https://tldr.sh

Built-in help: `git <command> --help` and `man <command>`

AI use declaration

Claude AI (Anthropic) was used to assist with the HTML layout and formatting structure of this document. All command descriptions, examples, and explanations were written by the author. The author takes full responsibility for the accuracy and completeness of the content presented in this cheat sheet.